



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO
FACULTAD DE INGENIERÍA
DIVISIÓN DE INGENIERÍA ELÉCTRICA
INGENIERÍA EN COMPUTACIÓN
LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



TAREA N° 04

NOMBRE COMPLETO: González Armas Dylan Arturo

N° de Cuenta: 419049513

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE 2027-1

FECHA DE ENTREGA LÍMITE: 1/10/2026

CALIFICACIÓN: _____

REPORTE DE TAREA

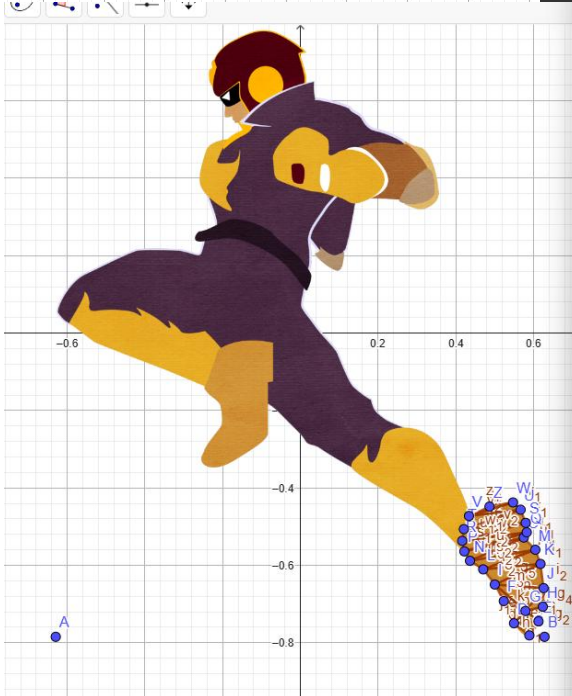
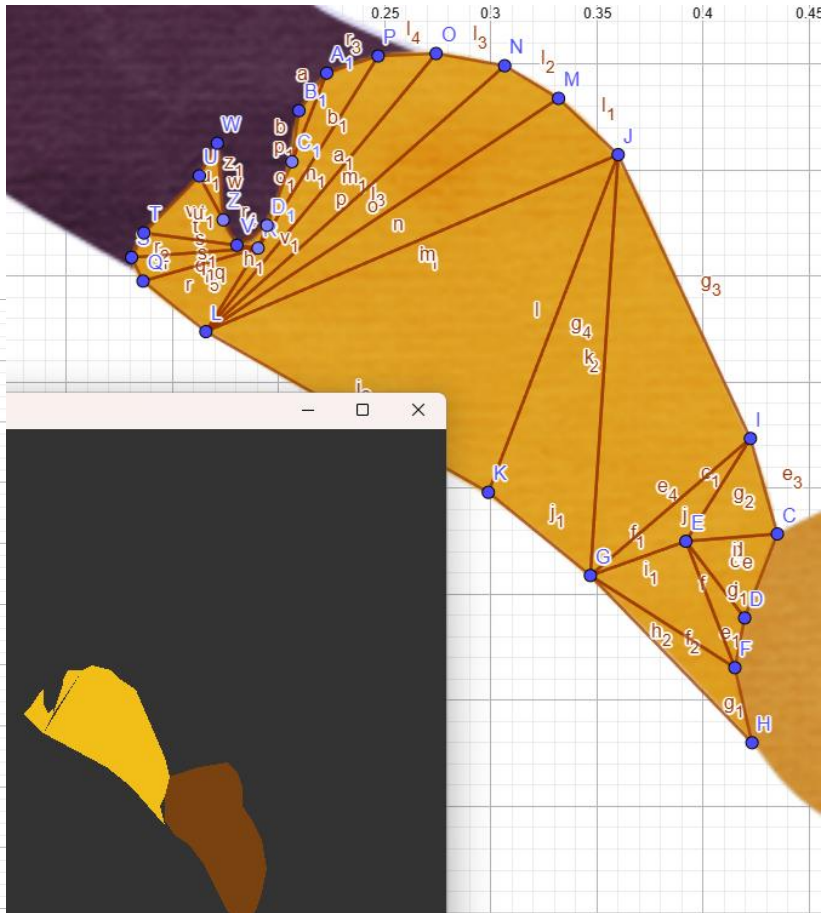
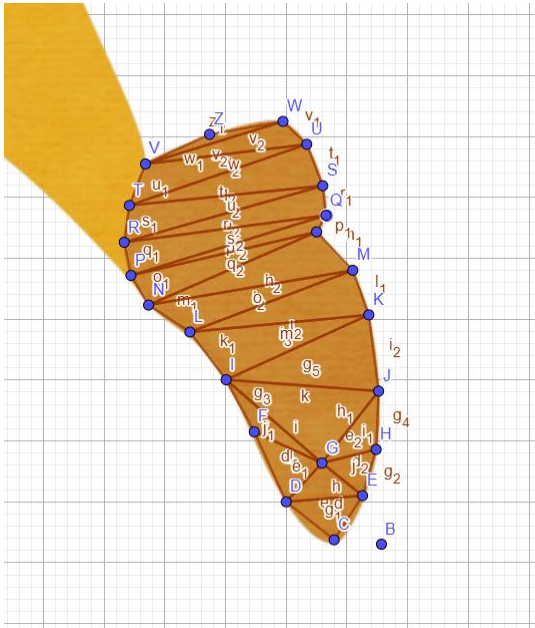
Para desarrollo de este trabajo utilice geogebra y realice manualmente la malla de triángulos sobre el dibujo, geogebra permite exportar coordenadas. Entonces mi flujo de trabajo fue producir la malla manualmente, exportarla como un archivo .html solicitar a una IA que convirtiera las coordenadas a formato GL_VERTEX_ARRAY y dibujar en entorno

capFalconCasco

Nombre	Definición	Valor
Punto A		A = (-0.63, -0.78)
Punto B		B = (0.63, -0.78)
Imagen imagen1		imagen1
Punto C		C = (-0.04, 0.62)
Punto D		D = (0.02, 0.64)
Punto E		E = (-0.02, 0.65)
Triángulo t1	Polígono(C, D, E)	t1 = 0
Segmento e	Segmento(C, D, t1)	e = 0.06
Segmento c	Segmento(D, E, t1)	c = 0.04
Segmento d	Segmento(E, C, t1)	d = 0.04
Punto F		F = (-0.08, 0.63)
Triángulo t2	Polígono(E, F, C)	t2 = 0
Segmento c ₁	Segmento(E, F, t2)	c ₁ = 0.06
Segmento e ₁	Segmento(F, C, t2)	e ₁ = 0.04
Segmento f	Segmento(C, E, t2)	f = 0.04
Punto G		G = (-0.1, 0.59)
Triángulo t3	Polígono(C, G, F)	t3 = 0
Segmento f ₁	Segmento(C, G, t3)	f ₁ = 0.06
Segmento c ₂	Segmento(G, F, t3)	c ₂ = 0.04
Segmento g	Segmento(F, C, t3)	g = 0.04
Punto H		H = (-0.15, 0.59)
Triángulo t4	Polígono(F, H, G)	t4 = 0
Segmento g ₁	Segmento(F, H, t4)	g ₁ = 0.08
Segmento f ₂	Segmento(H, G, t4)	f ₂ = 0.05
Segmento h	Segmento(G, F, t4)	h = 0.04
Punto I		I = (-0.13, 0.56)
Triángulo t5	Polígono(G, I, H)	t5 = 0
Segmento h ₁	Segmento(G, I, t5)	h ₁ = 0.04
Segmento g ₂	Segmento(I, H, t5)	g ₂ = 0.03
Segmento i	Segmento(H, G, t5)	i = 0.05
Punto J		J = (-0.16, 0.61)
Triángulo t6	Polígono(H, J, F)	t6 = 0
Segmento f ₃	Segmento(H, J, t6)	f ₃ = 0.02

capFalconBufanda

Nombre	Definición	Valor
Punto A		A = (-0.63, -0.78)
Punto B		B = (0.63, -0.78)
Imagen imagen1		imagen1
Punto C		C = (-0.19, 0.35)
Punto D		D = (-0.18, 0.32)
Punto E		E = (-0.21, 0.33)
Triángulo t1	Polígono(C, D, E)	t1 = 0
Segmento e	Segmento(C, D, t1)	e = 0.04
Segmento c	Segmento(D, E, t1)	c = 0.03
Segmento d	Segmento(E, C, t1)	d = 0.03
Punto F		F = (-0.18, 0.37)
Punto G		G = (-0.19, 0.38)
Triángulo t2	Polígono(C, F, G)	t2 = 0
Segmento g	Segmento(C, F, t2)	g = 0.02
Segmento c ₁	Segmento(F, G, t2)	c ₁ = 0.01
Segmento f	Segmento(G, C, t2)	f = 0.03
Punto H		H = (-0.17, 0.41)
Punto I		I = (-0.2, 0.41)
Triángulo t3	Polígono(G, H, I)	t3 = 0
Segmento i	Segmento(G, H, t3)	i = 0.04
Segmento g ₁	Segmento(H, I, t3)	g ₁ = 0.03
Segmento h	Segmento(I, G, t3)	h = 0.03
Punto J		J = (-0.16, 0.44)
Triángulo t4	Polígono(H, J, I)	t4 = 0
Segmento i ₁	Segmento(H, J, t4)	i ₁ = 0.03
Segmento h ₁	Segmento(J, I, t4)	h ₁ = 0.05
Segmento j	Segmento(I, H, t4)	j = 0.03
Punto K		K = (-0.16, 0.47)
Punto L		L = (-0.17, 0.44)
Triángulo t5	Polígono(J, K, L)	t5 = 0
Segmento l	Segmento(J, K, t5)	l = 0.04
Segmento j ₁	Segmento(K, L, t5)	j ₁ = 0.03
Segmento k	Segmento(L, J, t5)	k = 0.01



```

// EXPORTACIÓN DE VÉRTICES DE LA BOTA DERECHA (GeoGebra)
// =====
void CrearBotaDerecha()
{
    // Selección del color por defecto para la bota
#define R Colores::CAFE_BOTA[0]
#define G Colores::CAFE_BOTA[1]
#define B Colores::CAFE_BOTA[2]

    // Arreglo de vértices: X, Y, Z, R, G, B (21 Triángulos = 63 vértices = 378 floats)
    GLfloat vertices_bota_Der[] = {
        // Triángulo t1 (C, D, E)
        0.59f, -0.78f, 0.0f,  R, G, B,
        0.55f, -0.75f, 0.0f,  R, G, B,
        0.61f, -0.74f, 0.0f,  R, G, B,

        // Triángulo t2 (D, F, G)
        0.55f, -0.75f, 0.0f,  R, G, B,
        0.52f, -0.69f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,

        // Triángulo t3 (E, G, D)
        0.61f, -0.74f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,
        0.55f, -0.75f, 0.0f,  R, G, B,

        // Triángulo t4 (E, H, G)
        0.61f, -0.74f, 0.0f,  R, G, B,
        0.62f, -0.71f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,

        // Triángulo t5 (F, I, G)
        0.52f, -0.69f, 0.0f,  R, G, B,
        0.50f, -0.65f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,

        // Triángulo t6 (H, J, G)
        0.62f, -0.71f, 0.0f,  R, G, B,
        0.63f, -0.66f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,

        // Triángulo t7 (J, I, G)
        0.63f, -0.66f, 0.0f,  R, G, B,
        0.50f, -0.65f, 0.0f,  R, G, B,
        0.58f, -0.72f, 0.0f,  R, G, B,
    };
}

```

```

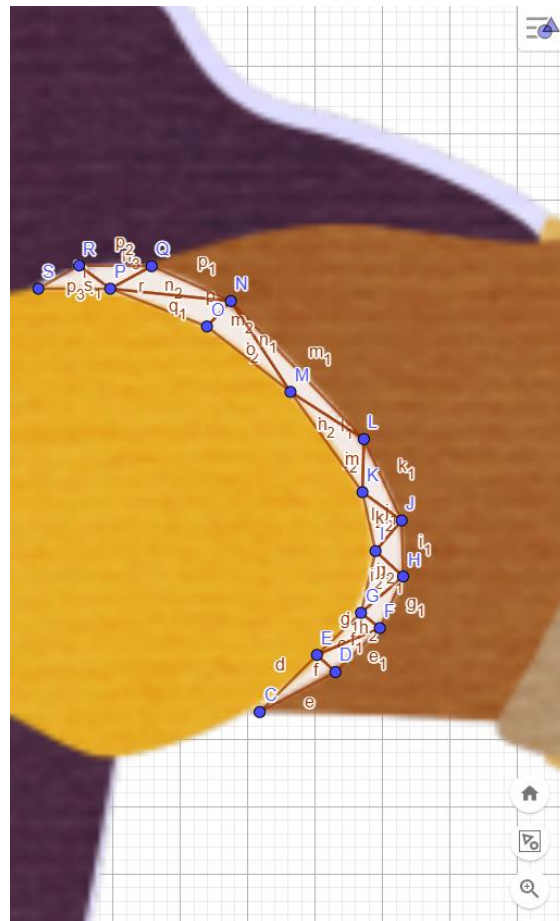
// Renderizado de la bota derecha
glm::mat4 model(1.0f);
model = glm::mat4(1.0f);
model = glm::translate(model, glm::vec3(0.0f, 0.0f, -1.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
meshColorList[0]->RenderMeshColor();

```

```

namespace Colores {
    const float CAFE_BOTA[3] = { 0.824f, 0.584f, 0.220f };
    const float CAFE_CAMISA[3] = { 0.478f, 0.255f, 0.067f };
    const float ROJO[3] = { 0.850f, 0.100f, 0.100f };
    const float DORADO[3] = { 0.950f, 0.750f, 0.100f };
    const float AMARILLO[3] = { 1.000f, 1.000f, 0.000f };
    const float MORADO[3] = { 0.300f, 0.180f, 0.310f };
    const float MORADO_CINTURON[3] = { 0.150f, 0.070f, 0.150f };
    const float PIEL[3] = { 0.860f, 0.610f, 0.310f };
    const float BLANCO[3] = { 1.000f, 1.000f, 1.000f };
    const float GRIS_OSCURO[3] = { 0.200f, 0.200f, 0.200f };
    const float MARRON_CODERA[3] = { 0.72f, 0.58f, 0.35f };
    const float MARRON_CODERA_INT[3] = { 0.712f, 0.583f, 0.447f };
    const float ROJO_OSCURO[3] = { 0.302f, 0.000f, 0.055f };
    const float NEGRO[3] = { 0.0f, 0.0f, 0.0f };
}

```

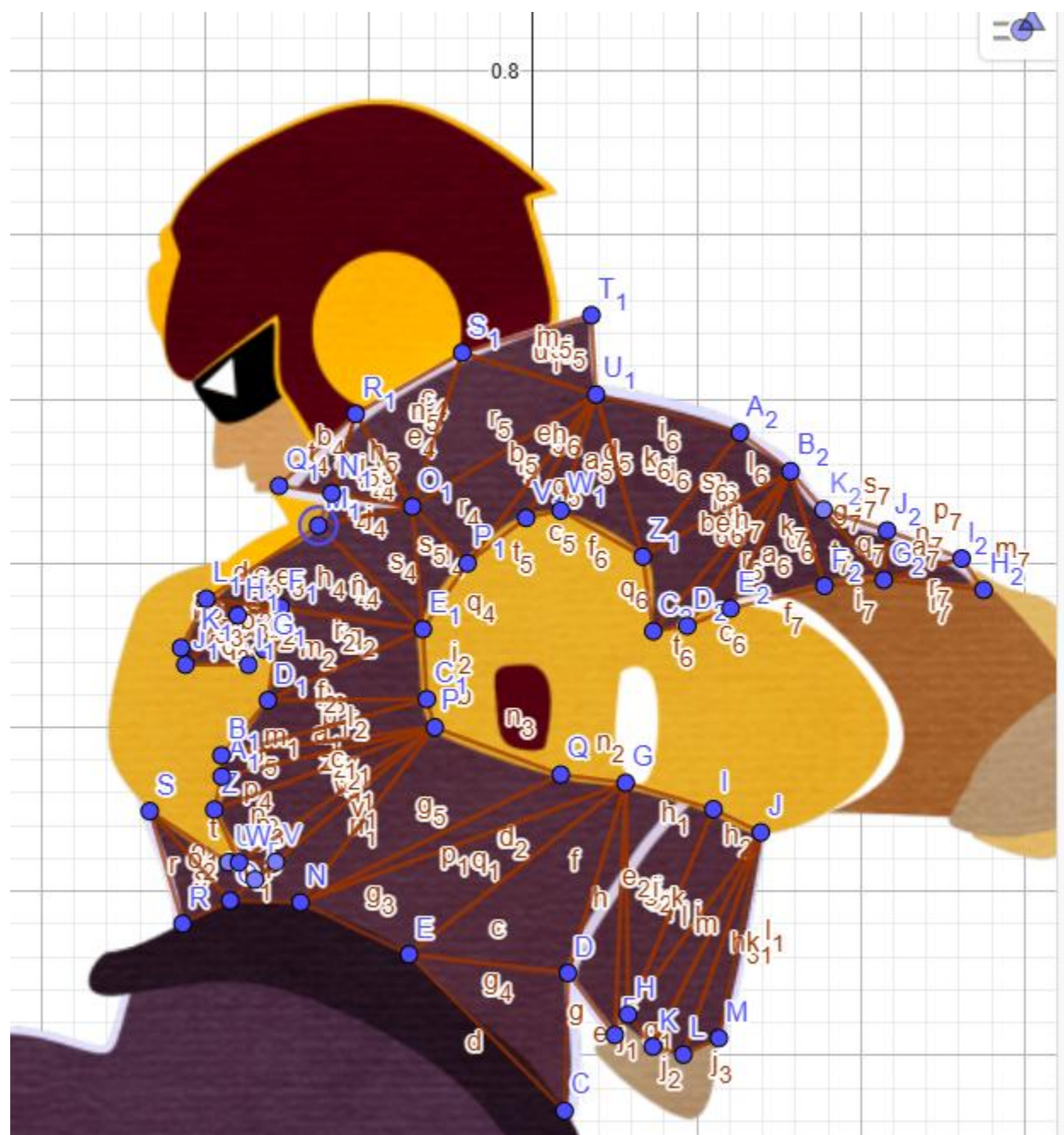


	Nombre	Descripción	Valor
1	Punto A		$A = (-0.63, -0.78)$
2	Punto B		$B = (0.63, -0.78)$
3	Imagen imagen1		imagen1
4	Punto C		$C = (0.18, 0.35)$
5	Punto D		$D = (0.21, 0.36)$
6	Punto E		$E = (0.2, 0.37)$
7	Triángulo t1	Polígono C, D, E	$t1 = 0$
7	Segmento e	Segmento [C, D]	$e = 0.03$
7	Segmento c	Segmento [D, E]	$c = 0.01$
7	Segmento d	Segmento [E, C]	$d = 0.02$
8	Punto F		$F = (0.22, 0.37)$
9	Triángulo t2	Polígono D, F, E	$t2 = 0$
9	Segmento e1	Segmento [D, F]	$e1 = 0.02$
9	Segmento d1	Segmento [F, E]	$d1 = 0.02$
9	Segmento f	Segmento [E, D]	$f = 0.01$
10	Punto G		$G = (0.21, 0.38)$
11	Triángulo t3	Polígono E, G, F	$t3 = 0$
11	Segmento f1	Segmento [E, G]	$f1 = 0.02$
11	Segmento e2	Segmento [G, F]	$e2 = 0.01$

```

CreateShaders(); // Carga shaderpersonaje.vert y shaderpersonaje.frag en shaderList[0]
CrearBotaDerecha(); // meshColorList[0]
CrearBotaIzquierda(); // meshColorList[1]
crearEspinilleraDer(); // meshColorList[2]
crearEspinilleraIzq(); // meshColorList[3]
CrearPiernas(); // meshColorList[4]
CrearCinturon(); // meshColorList[5]
CrearTorso(); // meshColorList[6]
CrearBufanda(); // meshColorList[7]
CrearCoderaExterna(); // meshColorList[8]
CrearCoderaInterna(); // meshColorList[9]
CrearCamisa(); // meshColorList[10]
CrearManga(); // meshColorList[11]
CrearDetalleGuanteRojo(); // meshColorList[12]
CrearDetalleGuanteBlanco(); // meshColorList[13]
CrearGuante(); // meshColorList[14]
CrearDetallesCasco(); // meshColorList[15]
CrearCasco(); // meshColorList[16]
CrearOjo(); // meshColorList[17]
CrearVisor(); // meshColorList[18]
CrearRostro(); // meshColorList[19]
CrearCoderaInterna2(); // meshColorList[20]
CrearSmashSupIzq(); // meshColorList[21]
CrearSmashSupDer(); // meshColorList[22]
CrearSmashInfIzq(); // meshColorList[23]
CrearSmashInfDer(); // meshColorList[24]

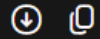
```

Código con la estructura de X, Y, Z, R, G, B (sin simetría/espeje)

Aquí tienes la traducción directa de los 21 triángulos (t_1 a t_{21}) del archivo GeoGebra respetando exactamente el mismo estilo de `CrearBotaDerecha()` :

C++

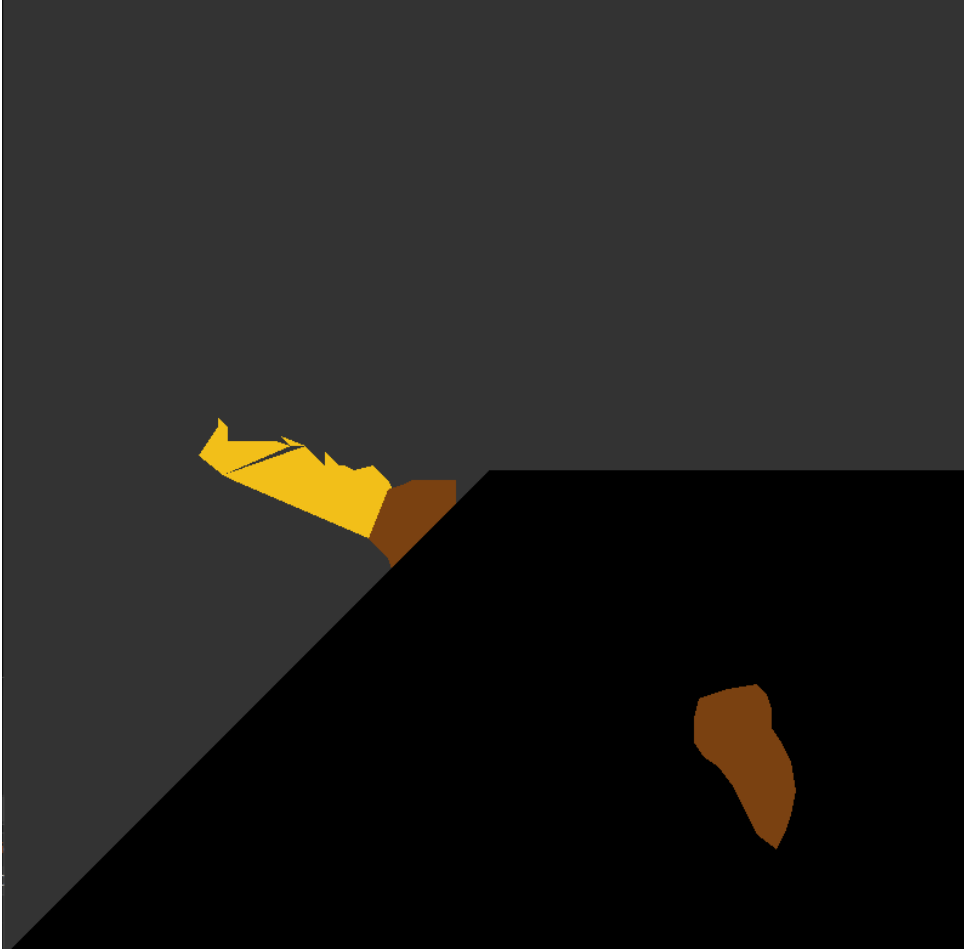


```
// =====  
// EXPORTACIÓN DE VÉRTICES DE LA ESPINILLERA (GeoGebra)  
// =====  
void CrearEspinillera()  
{  
    // Selección del color por defecto para la espinillera  
    #define R Colores::BLANCO[0]  
    #define G Colores::BLANCO[1]  
    #define B Colores::BLANCO[2]  
  
    // Arreglo de vértices: X, Y, Z, R, G, B (21 Triángulos = 63 vértices = 378 float)  
    GLfloat vertices_espinillera[] = {  
        // Triángulo t1 (C, D, E)  
        0.43f, -0.47f, 0.0f,    R, G, B,  
        0.42f, -0.51f, 0.0f,    R, G, B,  
        0.39f, -0.47f, 0.0f,    R, G, B,  
  
        // Triángulo t2 (D, F, E)  
        0.42f, -0.51f, 0.0f,    R, G, B,  
        0.41f, -0.53f, 0.0f,    R, G, B,  
        0.39f, -0.47f, 0.0f,    R, G, B,  
    }  
}
```

COMPARATIVA:



Problemas encontrados



Tuve problemas con el conteo de vértices que se tenían que dibujar. Usualmente, dependiendo del número de flotantes que se manden a la terminal se dibujan una cierta cantidad de vértices. Se debe ajustar este número correctamente para lograr un buen resultado al ejecutar el código.

3.- Comentario

Me encanto haber realizado este ejercicio, aprendí sobre formas en la que puedo trabajar OpenGL en el formato 2D usando la perspectiva ortogonal previamente revisada. Quisiera haber tenido mejor organización para tener un trabajo más satisfactorio.

Conclusión

Esta tarea me permitió familiarizarme más con el modo de trabajo OpenGL en perspectiva ortogonal para hacer dibujo 2D.

*Nota post entrega: Lo termine por puro gusto.

